Concurrency: Mutual Exclusion and Synchronization

Chapter 5

Concurrency Overview

- The central themes of operating system design are all concerned with the management of processes and threads:
  - Multiprogramming
  - Multiprocessing
  - Distributed processing
- Fundamental to operating system design is concurrency(并发)
- The Basic requirement for support of concurrency is the ability to enforce mutual exclusion(互斥)

Concurrency Contexts(并发上下文)

- Concurrency arises in three different contexts:
  - Multiple applications
    - Multiprogramming
  - Structured application
    - Application can be a set of concurrent processes or threads
  - Operating-system structure
    - Operating system is a set of processes or threads

Design Issues of Concurrency

- Communication among processes
- Sharing and competing(竞争) of resources
- Synchronization of the activities of multiple processes
- Allocation of processor time to processes

Principles of Concurrency

- Interleaving
- Overlapping
- Basic characteristic of multiprogramming systems: relative speed of execution of processes cannot be predicted

Difficulties of Concurrency

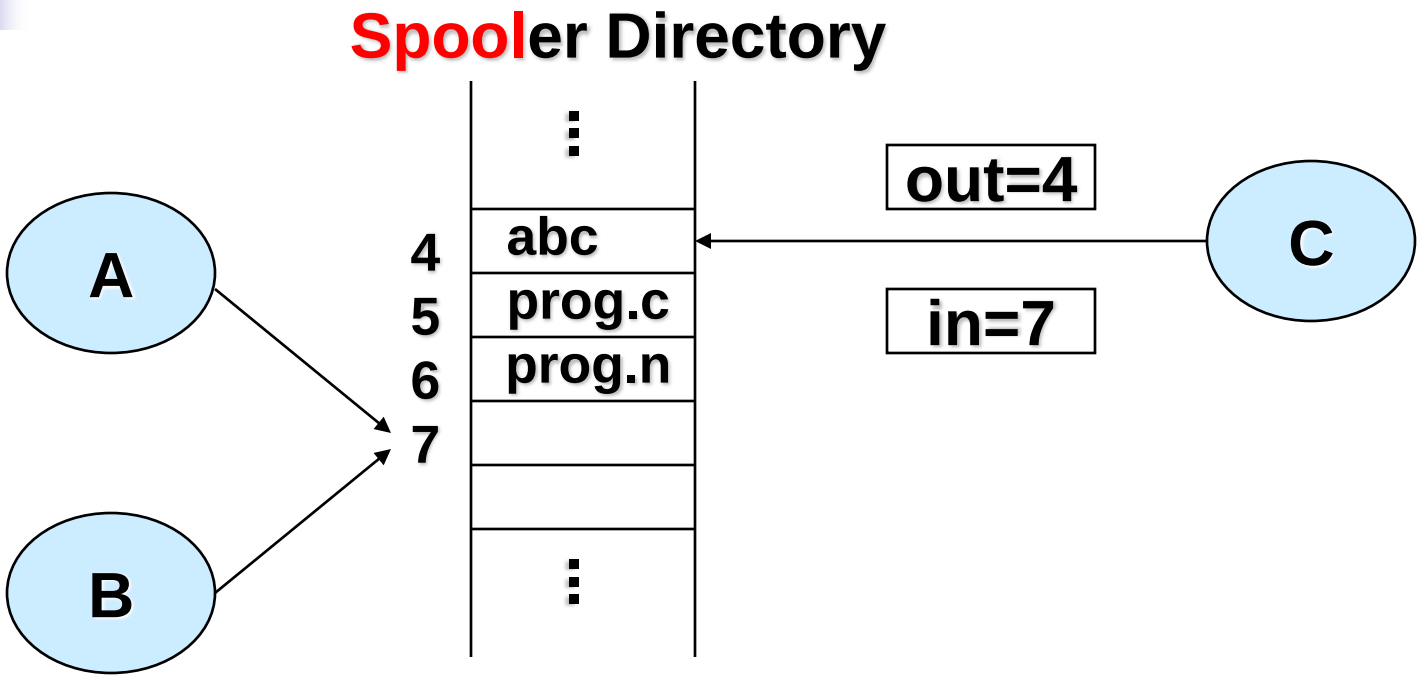
- Sharing of global resources
  - For example: Global variable
- Operating system managing the allocation of resources optimally
  - For example: I/O channel
- Difficult to locate programming errors
  - Program results are not deterministic and reproducible(再现)

A Simple Example

```
void echo()
{
  chin = getchar();
  chout = chin;
  putchar(chout);
}
```

```
  Process P1              Process P2
.                         .
chin = getchar();         .
.                         chin = getchar();
chout = chin;             chout = chin;
putchar(chout);                 .
.                         putchar(chout);
.                               .
```

Shared Memory Access (1)



- out - next file will be printed
- in - next free slot
- c - printer daemon(打印机守护进程)

Shared Memory Access (2)

- Process A read in=7, save in next_free_slot, clock interrupt occurs, process A is been switched
- Process B is scheduled to run, B read in=7, save the file to slot 7, update in=8, and continue to run
- Process A is restored to run, it checks next_free_slot=7, and save the file to slot 7, next_free_slot +1=>in, then in=8。
- The file which process B saved in slot 7 is lost，and printer daemon would not find this error

Serial(串行) and Parallel(并行)
Concurrent(并发)

Race Condition(竞争条件)

- When multiple processes or threads access shared resource so that the final result depends on the order of execution of instructions in the multiple processes
- Murphy Law
  - If Something can go wrong, it will.
- Solution to Race Condition:
  - Control access to the shared resource
- Mutual Exclusion(互斥)

Process Interaction

- Three possible degrees of awareness among processes:
  - Processes unaware of each other
    - Competition
  - Processes indirectly aware of each other
    - Cooperation
  - Process directly aware of each other
    - Cooperation

Competition Among Processes for Resources

- Critical sections(临界区)
  - Portion of the program(instructions) that access shared resource
    - Only one program at a time is allowed in its critical section
- Enforcement of mutual exclusion incurs two additional problems:
  - Deadlock(死锁)
  - Starvation(饥饿)

Mutual Exclusion Code Skeleton

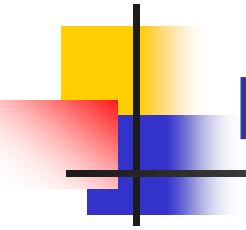
```
/*program mutual exclusion */
const int n=/*number of processes*/;

void P(int i)
{
   while (true)
  {
        …                          /*noncritical section*/
        entercritical(i);
        …                          /*critical section/region*/
        exitcritical(i);
        …                         /*remainder, noncritical section*/
   }
}

void main()
{
   parbegin(P(1), P(2)…, P(n));
}
```

Requirements for Mutual Exclusion

- Only one process at a time is allowed in the critical section for a resource
- A process must not be delayed to enter a critical section when there is no other process accesses the same resource
- A process that halts in its noncritical section must do so without interfering with other processes
- No assumptions are made about relative process speeds or number of processes
- A process remains inside its critical section for a finite time only
- No deadlock or starvation



Lock Variables(锁变量)

- The simplest approach we can imagine
- How does it work?

Lock Variables(锁变量)

```
    …                          ;noncritical section
entercritical:
    cmp  lock, 0
    jnz    entercritical
    mov  lock, 1
    …                          ;critical setion
exitcritical:
    mov  lock, 0
    …                          ;noncritical section
```

Mutual Exclusion:Hardware Support

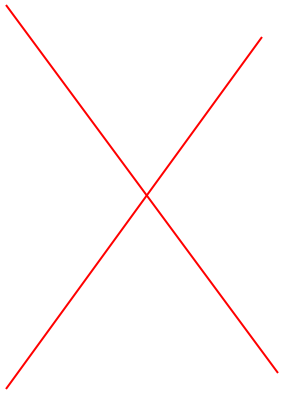
- Interrupt Disabling(关中断)
  - A process runs until it invokes an operating system service or until it is interrupted
  - Disabling interrupts guarantees mutual exclusion
  - Processor is limited in its ability to interleave programs
  - Multiprocessing
    - disabling interrupts on one processor will not guarantee mutual exclusion

Mutual Exclusion:Hardware Support

- Special Machine Instructions
  - Performed in a single instruction cycle
  - Access to the memory location is blocked for any other instructions
- Test and Set Instruction
- Exchange Instruction

Test and Set Instruction

```
boolean testset (int i)
{
  if (i == 0) {
     i = 1;
     return true;
  }
  else {
     return false;
  }
}
```

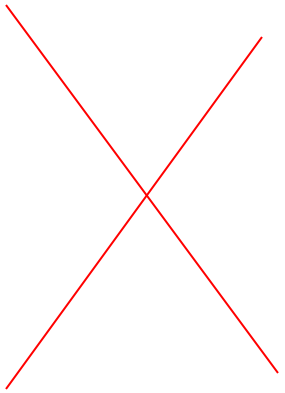


Test and Set Instruction

```
    …                                ;noncritical section
entercritical:
    ts      reg, lock
    cmp    reg, 0
    jnz     entercritical
    …                                ;critical setion
exitcritical:
    mov  lock, 0
    …                                ;noncritical sectio
```

Exchange Instruction

```
void exchange(int register,
     int memory)
{
  int temp;
  temp = memory;
  memory = register;
  register = temp;
}
```



Exchange Instruction

```
    …                                ;noncritical section
entercritical:
    mov  reg, 1
    xchg  reg, lock
    cmp   reg, 0
    jnz    entercritical
    …                                ;critical setion
exitcritical:
    mov  lock, 0
    …                                ;noncritical sectio
```

Test and Set for Mutual Exclusion

```
/*program mutualexclusion*/
const int n = /* number of processes */;
int bolt;
void P(int i)
{
     while (true)
     {
          while (!testset (bolt)) ;  /* do nothing */
          /* critical section */;
          bolt = 0;
          /* remainder */
     }
}
```

```
void main()
{
    bolt = 0;
    parbegin (P(1), P(2), . . . ,P(n));
}
```

Exchange for Mutual Exclusion

```
/*program mutualexclusion*/
const int n = /* number of processes */;
int bolt;
void P(int i)
{
    int keyi = 1;
    while (true)
     {
         do exchange(keyi, bolt)  while (keyi != 0);
         /* critical section */;
         exchange(keyi, bolt);
         /* remainder */
     }
}
```

```
void main()
{
    bolt = 0;
    parbegin (P(1), P(2), . . . ,P(n));
}
```

Exchange for Mutual Exclusion

```
/*program mutualexclusion*/
const int n = /* number of processes */;
int bolt;
void P(int i)
{
    int keyi = 1;
    while (true)
     {
        while (keyi != 0) exchange(keyi, bolt);
         /* critical section */;
         exchange(keyi, bolt);
         /* remainder */
     }
}
```

```
void main()
{
    bolt = 0;
    parbegin (P(1), P(2), . . . ,P(n));
}
```

Advantages of Machine-instruction Approach

- Applicable to any number of processes on either a single processor or multiple processors sharing main memory
- It is simple and therefore easy to verify
- It can be used to support multiple critical sections

Disadvantages of Machine-instruction Approach

- Busy-waiting(忙等待) consumes processor time
- Starvation is possible when a process leaves a critical section and more than one process is waiting.
- Deadlock
  - If a low priority process has the critical region and a higher priority process needs, the higher priority process will obtain the processor to wait for the critical region
  - Priority Inversion Problem(优先级翻转问题)

Semaphores(信号量)

- 1965, Dijkstra
- Special variable called a semaphore is used for signaling
- Two primitives(原语) are defined:
  - Wait operation decrements the semaphore value
  - Signal operation increments semaphore value
  - P/V operations
  - Down/Up operations
- Two kinds of semaphores:
  - counting semaphores: can take on any non-negative value
  - binary semaphores: take on only the values 0 and 1.(Von a binary semaphore with value 1 has no effect.)

Semaphore Primitives

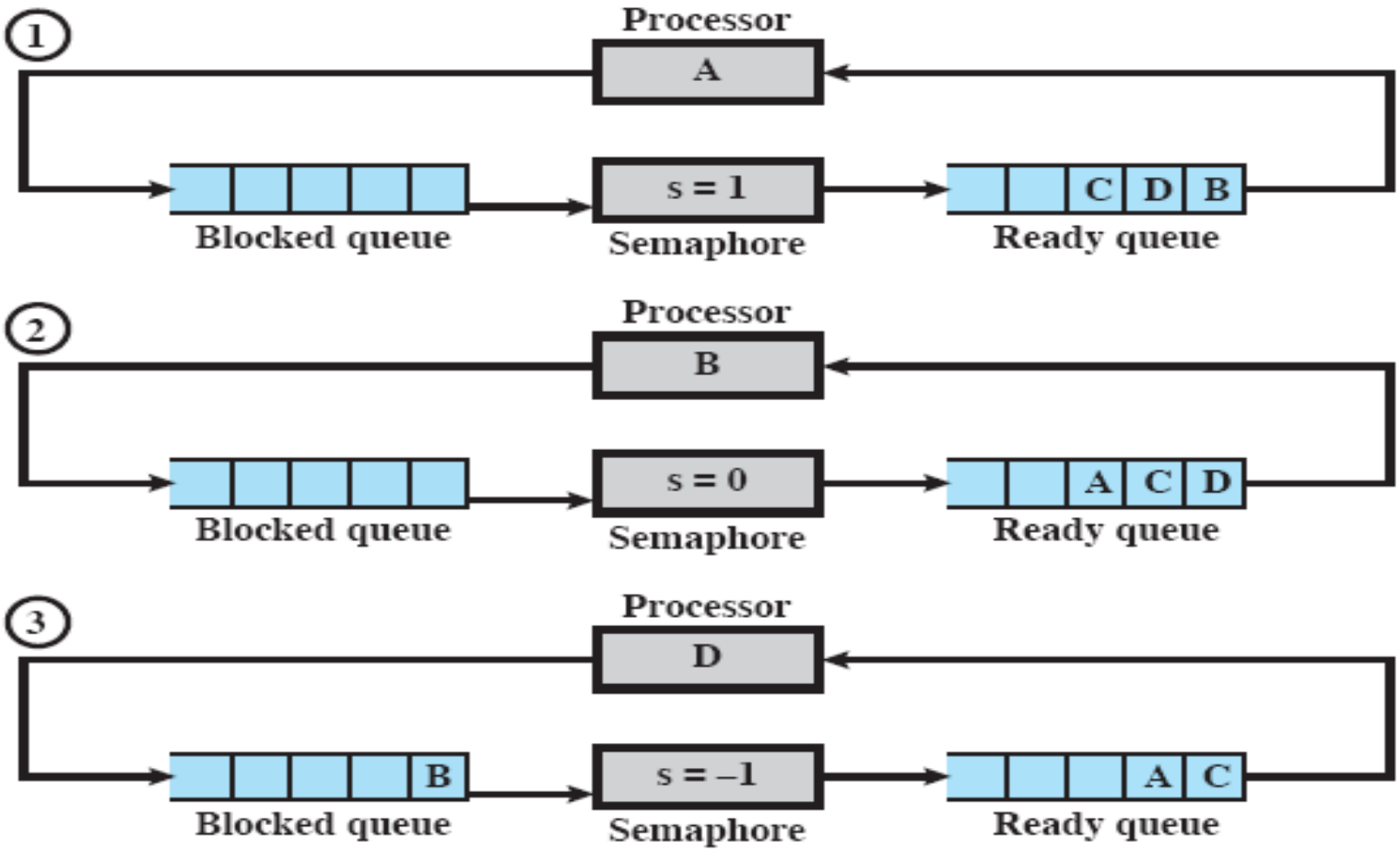
```
struct semaphore {
    int count;
    queueType queue;
}

void semWait (semaphore s)
{
    s.count--;
    if (s.count < 0)  {
        place this process in s.queue;
        block this process
    }
}
void semSignal (semaphore s)
{
    s.count++;
    if (s.count <= 0)  {
        remove a process P from s.queue;
        place process P on ready list;
    }
}
```

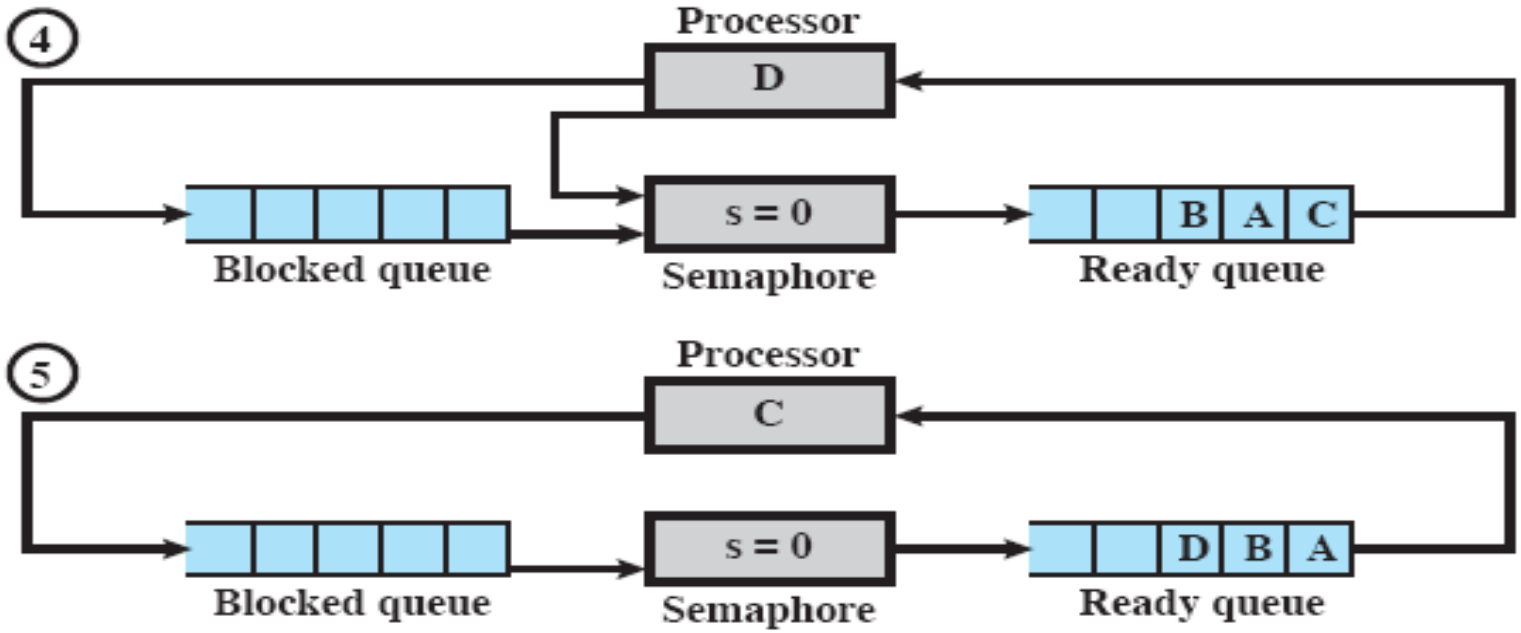
Example of Semaphore Mechanism

- Processes A, B and C depend on the result of process D
- Initially semaphore count is 1, indicating that only one of D's results is available

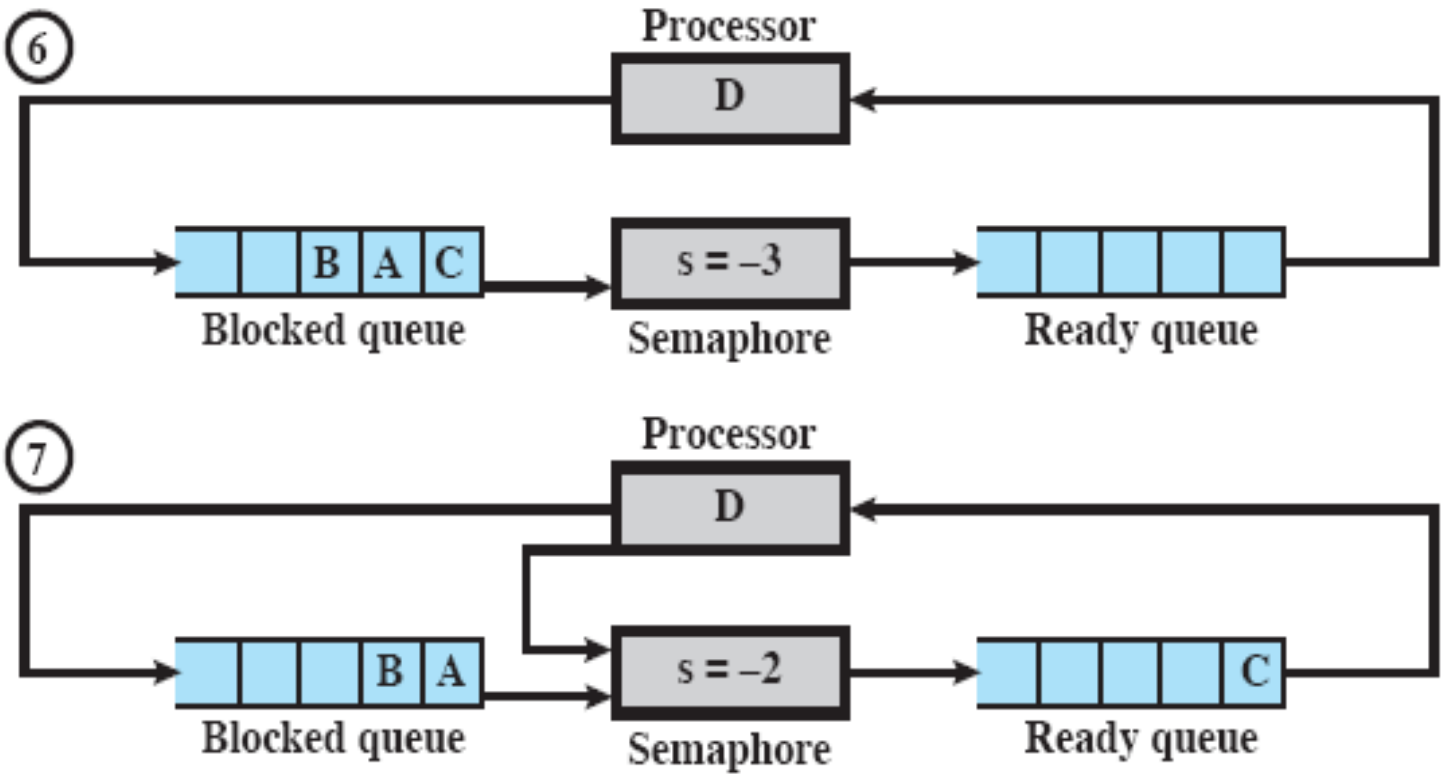
Example of Semaphore Mechanism



Example of Semaphore Mechanism



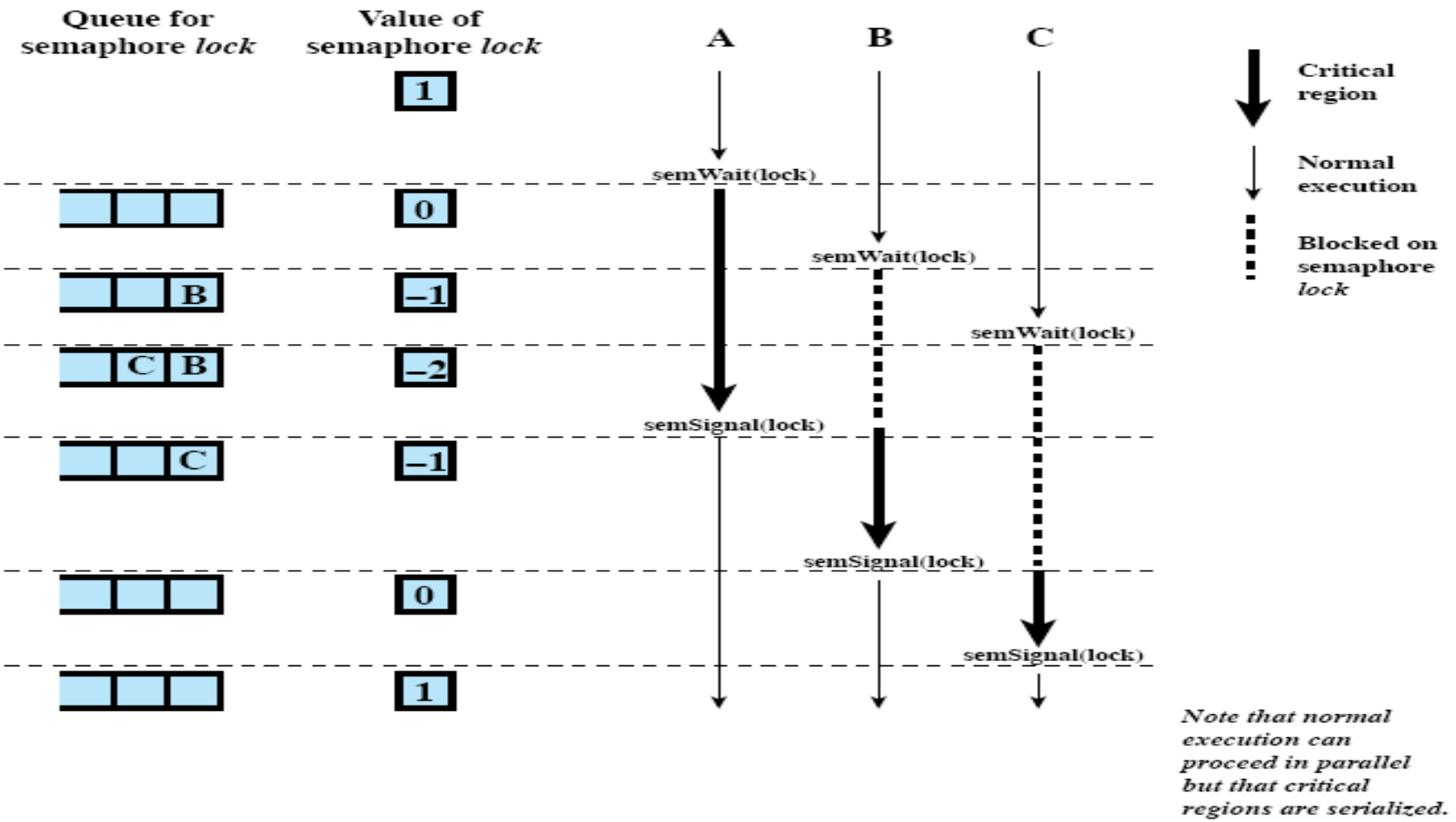
Example of Semaphore Mechanism



Mutual Exclusion Using Semaphores

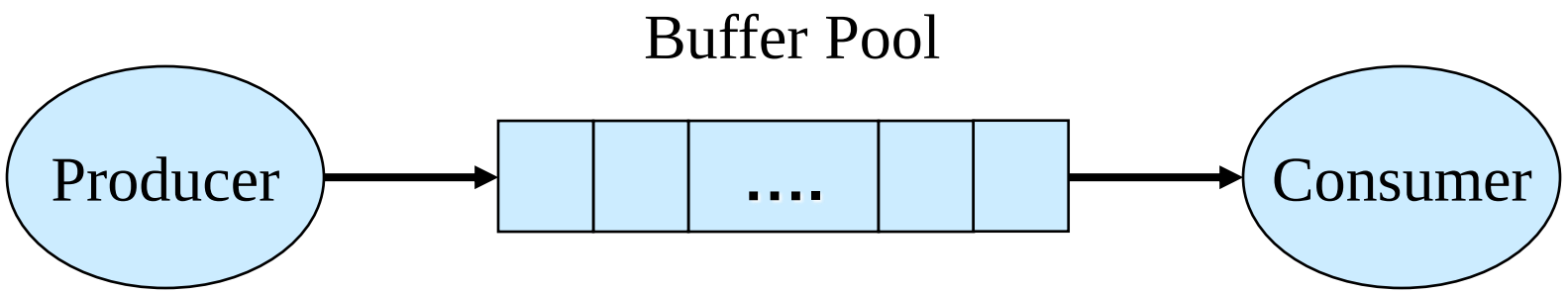
```
/* program mutual exclusion */
const int n = /* number of processes */;
semaphore mutex = 1;
void P (int i)
{
    while (true)
    {
        …                              /* noncritical section */
        semWait(mutex);
        …                              /* critical section */;
        semSignal(mutex);
        …                              /* remainder, noncritical section */;
    }
}
void main()
{
    parbegin (P(1), P(2), . . ., P(n));
}
```

Processes Accessing Shared Data Protected by a Semaphore



Producer/Consumer Problem

- One or more producers are generating data and placing these in a buffer
- A single consumer is taking items out of the buffer one at time
- Only one producer or consumer may access the buffer at any one time



Producer/Consumer Problem

```
#define  N 100
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;
```

```
void producer(void)
{
    int item;
    while(true){
        produce_item(&Item);
        semWait(&empty);
        semWait(&mutex);
        enter_item(item);
        semSignal(&mutex);
        semSignal(&full);
    }
}
```

```
void consumer(void)
{
    int item;
    while(true){
        semWait(&full);
        semWait(&mutex);
        remove_item(&item);
        semSignal(&mutex);
        semSignal(&empty)
        consume_item(item);
    }
}
```

Producer/Consumer Problem

```
#define  N 100
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;
```

```
void producer(void)
{
    int item;
    while(true){
        produce_item(&Item);
        semWait(&mutex);
        semWait(&empty);
        enter_item(item);
        semSignal(&mutex);
        semSignal(&full);
    }
}
```

```
void consumer(void)
{
    int item;
    while(true){
        semWait(&full);
        semWait(&mutex);
        remove_item(&item);
        semSignal(&mutex);
        semSignal(&empty)
        consume_item(item);
    }
}
```

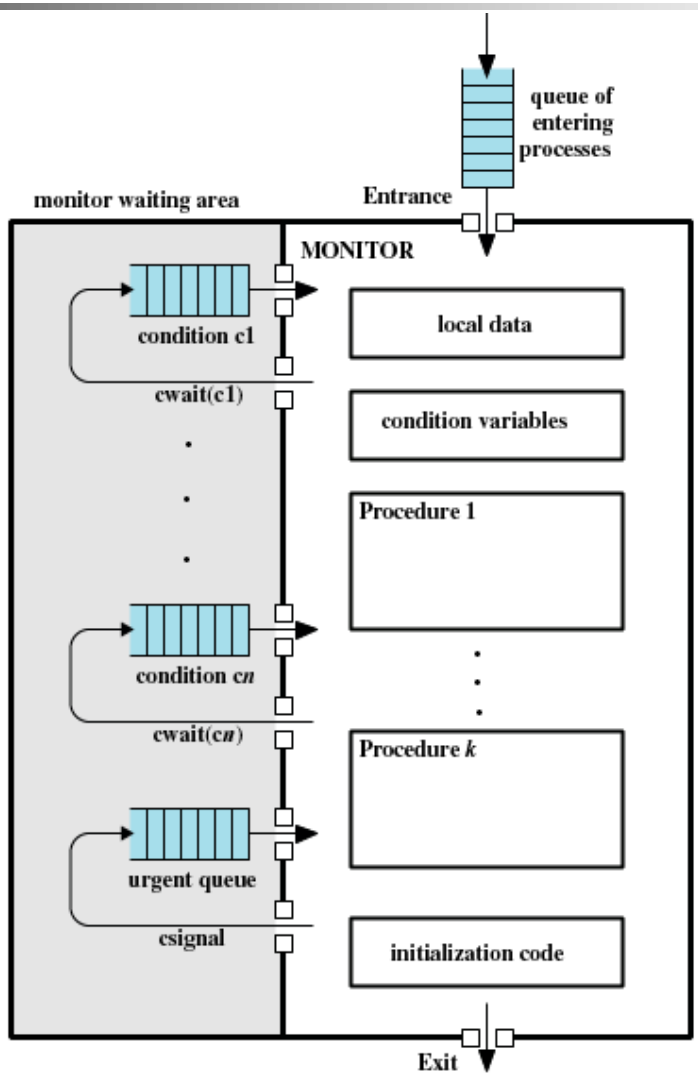
Monitors(管程)

- Monitor is a software module consisting of one or more procedures, an initialization sequence, and local data
- Chief characteristics
  - Local data variables are accessible only by the monitor's procedures and not by any external procedure
  - Process enters monitor by invoking one of its procedures
  - Only one process may be executing in the monitor at a time

Condition Variables

- Condition variable are a special data type in monitors, which are operated on by two operations:
  - cwait(c): suspend execution of the calling process on condition c. The monitor is now available for use by another process
  - csignal(c): resume execution of some process blocked after a cwait on the same condition. If there are several such processes, choose one of them; if there is no such process, do nothing.

Structure of a Monitor



Producer/Consumer Problem

```
/* program producerconsumer */
monitor boundedbuffer;
char buffer [N];                                        /* space for N items */
int nextin, nextout;                                      /* buffer pointers */
int count;                                        /* number of items in buffer */
cond notfull, notempty;          /* condition variables for synchronization */

void append (char x)
{
     if (count == N)
        cwait(notfull);                      /* buffer is full; avoid overflow */
     buffer[nextin] = x;
     nextin = (nextin + 1) % N;
     count++;
     /* one more item in buffer */
     csignal(notempty);                          /* resume any waiting consumer */
}
void take (char x)
{
     if (count == 0)
        cwait(notempty);                   /* buffer is empty; avoid underflow */
     x = buffer[nextout];
     nextout = (nextout + 1) % N;
     count--;                                       /* one fewer item in buffer */
     csignal(notfull);                          /* resume any waiting producer */
}
{                                                                 /* monitor body */
     nextin = 0; nextout = 0; count = 0;           /* buffer initially empty */
}
```

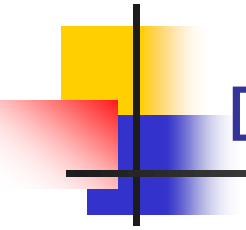
Producer/Consumer Problem

```
void producer()
char x;
{
     while (true)
     {
     produce(x);
     append(x);
     }
}
void consumer()
{
     char x;
     while (true)
     {
       take(x);
       consume(x);
     }
}
void main()
{
     parbegin (producer, consumer);
}
```

Figure 5.16 A Solution to the Bounded-Buffer Producer/Consumer Problem Using a Monitor

Message Passing(消息传递)

- Enforce mutual exclusion
- Exchange information
- A pair of primitives is provided for message passing:
  - **`send(destination, message)`**
  - **`receive(source, message)`**



Design Issues Relating to Message-passing

- Synchronization
- Addressing
- Message Format
- Queuing Discipline

Synchronization

- Sender and receiver may or may not be blocking (waiting for message)
- Blocking send, blocking receive
  - Both sender and receiver are blocked until message is delivered
  - Called a rendezvous (会合)
- Nonblocking send, blocking receive
  - Sender continues on
  - Receiver is blocked until the requested message arrives
- Nonblocking send, nonblocking receive
  - Neither party is required to wait

Addressing

- Direct addressing
  - Send primitive includes a specific identifier of the destination process
  - Receive primitive could know ahead of time which process a message is expected
  - Receive primitive could use source parameter to return a value when the receive operation has been performed

Addressing (cont.)

- Indirect addressing
  - Messages are sent to a shared data structure consisting of queues
  - Queues are called mailboxes
  - One process sends a message to the mailbox and the other process picks up the message from the mailbox

Indirect Process Communication

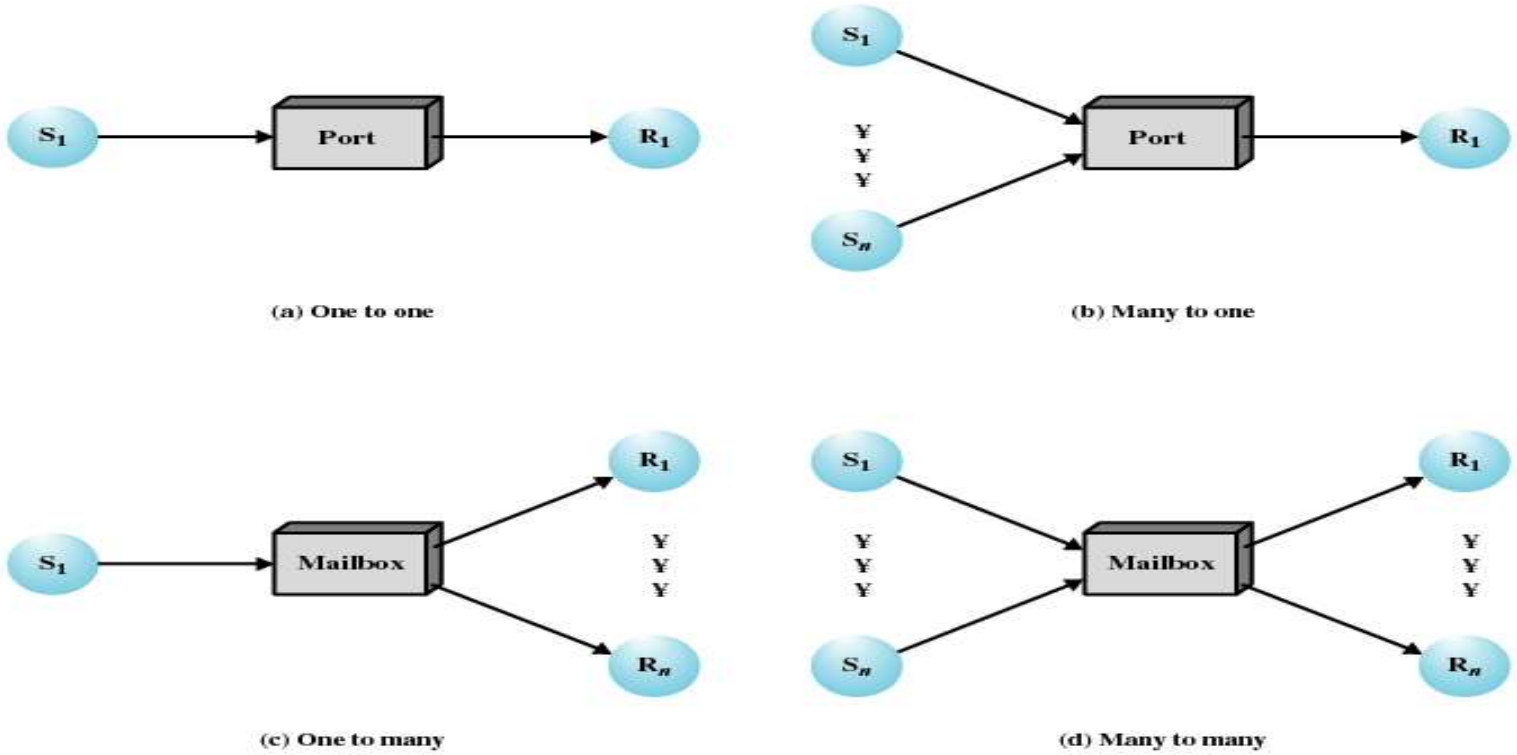


Figure 5.18 Indirect Process Communication

Message Format

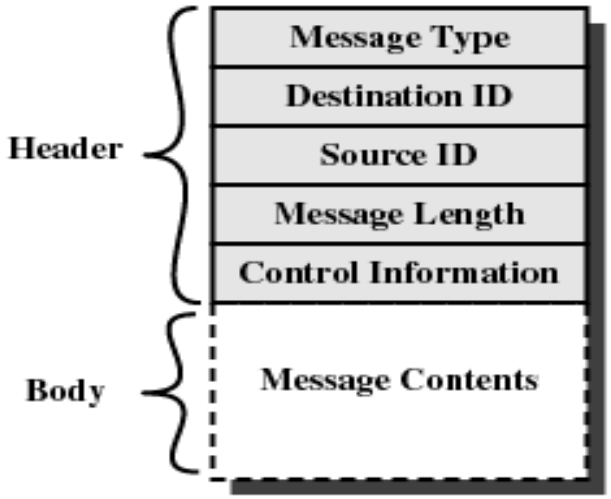


Figure 5.19 General Message Format

Queuing Discipline

- FIFO
- Priority-based strategies

Mutual Exclusion Using Messages

```
/* program mutualexclusion */
const int n = /* number of processes  */;
void P(int i)
{
    message msg;
    while (true)
    {
     receive (mutex, msg);
     /* critical section   */;
     send (mutex, msg);
     /* remainder    */;
    }
}
void main()
{
    create_mailbox (mutex);
    send (mutex, null);
    parbegin (P(1), P(2), . . ., P(n));
}
```

Figure 5.20 Mutual Exclusion Using Messages

Solution to Producer/Consumer

```
const int
     capacity = /* buffering capacity */ ;
     null =/* empty message */ ;
int i;
void producer()
{    message pmsg;
     while (true)
     {
       receive (mayproduce, pmsg);
       pmsg = produce();
       send (mayconsume, pmsg);
     }
}
void consumer()
{    message cmsg;
     while (true)
     {
       receive (mayconsume, cmsg);
       consume (cmsg);
       send (mayproduce, null);
     }
}

void main()
{
     create_mailbox (mayproduce);
     create_mailbox (mayconsume);
     for (int i = 1; i <= capacity; i++)
         send (mayproduce, null);
     parbegin (producer, consumer);
}
```

Figure 5.21 A Solution to the Bounded-Buffer Producer/Consumer Problem Using Messages

Readers/Writers Problem

- Any number of readers may simultaneously read the file
- Only one writer at a time may write to the file
- If a writer is writing to the file, no reader may read it

Readers/Writers Problem

```
/* program readersandwriters */
int readcount;
semaphore x = 1, wsem = 1;
void reader()
{
    while (true)
    {
     semWait (x);
     readcount++;
     if (readcount == 1)
          semWait (wsem);
     semSignal (x);
     READUNIT();
     semWait (x);
     readcount--;
     if (readcount == 0)
          semSignal (wsem);
     semSignal (x);
    }
 }
void writer()
{
    while (true)
    {
     semWait (wsem);
     WRITEUNIT();
     semSignal (wsem);
    }
}

void main()
{
    readcount = 0;
    parbegin (reader, writer);
    }
```

```
/*program readersandwriters*/
int  readcount, writecount;
semaphore x = 1, y = 1, z = 1, wsem = 1, rsem = 1;
void reader()
{
    while (true)
    {
     semWait (z);
          semWait (rsem);
               semWait (x);
                    readcount++;
                    if (readcount == 1)
                         semWait (wsem);
               semSignal (x);
          semSignal (rsem);
     semSignal (z);
     READUNIT();
     semWait (x);
          readcount--;
          if (readcount == 0)
               semSignal (wsem);
     semSignal (x);
    }
}
void writer ()
{
    while (true)
    {
     semWait (y);
          writecount++;
          if (writecount == 1)
               semWait (rsem);
     semSignal (y);
     semWait (wsem);
     WRITEUNIT();
     semSignal (wsem);
     semWait (y);
          writecount--;
          if (writecount == 0)
               semSignal (rsem);
     semSignal (y);
    }
}
void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}
```